

Understanding Impact of Coding Style on Synthesis Using Yosys

INTRODUCTION:

This experiment demonstrates how logic optimization in Yosys reduces redundant hardware resources while preserving functionality. The chosen example contains two multipliers in RTL, but only one is required at a time. Yosys optimization passes (opt, share) can detect this pattern and merge them into a single hardware multiplier. By generating two netlists (optimized vs unoptimized), students will observe the practical impact of resource sharing and logic simplification during synthesis.

OBJECTIVE:

After completing this experiment, you will be able to:

- Understand how resource sharing reduces hardware usage.
- Compare optimized vs unoptimized synthesis results.
- Use Yosys optimization passes (opt, share, etc.).
- Analyze stat reports to confirm area and logic savings.

PROCEDURE:

Step 1: Write Different Verilog code for decoder

1.1: Open gvim editor for design with command: `gvim decoder1.v`

```
module decoder_expr (
    input en,
    input [1:0] a,
    output [3:0] y
);

assign y[0] = en & ~a[1] & ~a[0];
assign y[1] = en & ~a[1] & a[0];
assign y[2] = en & a[1] & ~a[0];
assign y[3] = en & a[1] & a[0];

endmodule

~
```

1.2: Save the file using command- :wq!

1.3 Open Gvim again and write the next code gvim decoder2.v

```
module decoder_case (
    input en,
    input [1:0] a,
    output reg [3:0] y
);

always @(*) begin
    y = 4'b0000;
    if (en) begin
        case(a)
            2'b00: y = 4'b0001;
            2'b01: y = 4'b0010;
            2'b10: y = 4'b0100;
            2'b11: y = 4'b1000;
        endcase
    end
end
endmodule
```

1.4: Save the file using wq!

Step 2: Synthesize all the codes in Yosys with the following script

//Change the script as per the file

```
# Step 1: Read the Verilog source file
read_verilog decoder1.v

# Step 2: Set the top-level module and check hierarchy
hierarchy -check -top decoder_expr

# Step 3: High-level synthesis passes
proc          # process always blocks
opt           # basic optimizations

# Step 4: Map to internal cell library
techmap
opt

# Step 5: Map flip-flops to the standard cell library
dfflibmap -liberty osu018_stdcells.lib

# Step 6
stat

# Step 7: Cleanup unused cells and signals
clean

# Step 8: Write the synthesized gate-level netlist
write_verilog decoder1_synth.v

show
```

Step 3: After Synthesis of each code analyse the difference with the stat command

```
//decoder1.v - stat
```

```
8. Printing statistics.
```

```
=== decoder_expr ===
```

Number of wires:	7
Number of wire bits:	11
Number of public wires:	3
Number of public wire bits:	7
Number of memories:	0
Number of memory bits:	0
Number of processes:	0
Number of cells:	8
\$ _AND_	6
\$ _NOT_	2

```
//decoder2.v -stat
```

```
8. Printing statistics.
```

```
=== decoder_case ===
```

Number of wires:	15
Number of wire bits:	28
Number of public wires:	3
Number of public wire bits:	7
Number of memories:	0
Number of memory bits:	0
Number of processes:	0
Number of cells:	21
\$ _MUX_	8
\$ _NOT_	4
\$ _OR_	7
\$ _XOR_	2

OBSERVATION:

Behavioral (case) – decoder_case.v:

- Yosys infers a multiplexer from the case statement.
- Synthesized design is optimized and minimal.
- stat shows a small number of logic elements.

Dataflow (assign) – decoder_expr.v:

- Similar inference as behavioral.
- Slightly more compact code with identical synthesis output.
- Very suitable for small combinational circuits.

OUTPUT:

- `decoder_case.v` – Behavioral description using `case` statement.
- `decoder_expr.v` – Dataflow description using `assign` statements.
- Yosys Reports: Can be saved via `tee` command or copy-paste for documentation.

CONCLUSION:

This experiment demonstrated how different Verilog coding styles, though functionally identical, can influence how the synthesis tool maps them to hardware. While behavioral and dataflow descriptions lead to efficient and optimized designs, gate-level coding may result in larger netlists with more logic elements. Coding style matters not only for readability but also for synthesis quality, area, and design clarity.